React JS Notes: Conditional Rendering

Level: Beginner
Topic: Displaying UI Based on Conditions
Language: JavaScript / JSX

> **Core idea:** Conditional rendering means displaying different UI elements depending on whether a condition is true or false.

---

1. What Is Conditional Rendering?

Conditional rendering means displaying different UI elements based on a condition.

In simple words:

- If a condition is `true`, display something.
- If a condition is `false`, display something else or display nothing.

Real-Life Examples

- If a student passes, show **"Pass"**.
- If a user is logged in, show **"Logout"**.
- If a product is available, show **"Add to Cart"**.
- If data is loading, show **"Loading..."**.

---

2. Why Do We Need Conditional Rendering?

We use conditional rendering to:

1. Show or hide elements.
2. Display different messages.
3. Show Login or Logout buttons.
4. Display Admin or User dashboards.
5. Show loading and error messages.
6. Display different content based on user roles.

---

3. Common Methods of Conditional Rendering in React

The most common methods are:

1. `if-else` statement
2. Ternary operator (`? :`)
3. Logical AND operator (`&&`)
4. Logical OR operator (`||`)
5. Conditional rendering using variables
6. Multiple conditions using `if-else if` or nested ternary

7. Conditional rendering using `switch`
8. Conditional rendering with `map()`

Part 1: Basic Conditional Rendering

4. Conditional Rendering Using `if-else`

We can use normal JavaScript `if-else` statements to decide what a component should return.

Example

```
function App() {
    const marks = 75;

    if (marks >= 33) {
        return <h1>Pass</h1>;
    } else {
        return <h1>Fail</h1>;
    }
}

export default App;
```

Explanation

- If `marks` are 33 or greater, **Pass** is displayed.
- Otherwise, **Fail** is displayed.

Important: A normal `if-else` statement cannot be used directly inside JSX curly braces as an expression. Use it before the `return` statement or inside a function.

---

5. Conditional Rendering Using the Ternary Operator (`? :`)

The ternary operator is useful when we want to choose between two outputs.

Syntax

```
condition ? expression1 : expression2;
```

- `?` means **if true**.
- `:` means **otherwise**.

Example

```
function App() {
    const isAdmin = true;

    return (
        <div>
            {isAdmin ? <h1>Admin</h1> : <h1>User</h1>}
        </div>
    );
}

export default App;
```

Explanation

- If `isAdmin` is `true`, **Admin** is displayed.
- If `isAdmin` is `false`, **User** is displayed.

---

6. Conditional Rendering Using Logical AND (&&)

The `&&` operator is used when we want to display something only if a condition is true.

Syntax

```
condition && element
```

Example

```
function App() {
    const isLoggedIn = true;

    return (
        <div>
            <h1>Welcome</h1>

            {isLoggedIn && <h2>Welcome Back!</h2>}
        </div>
    );
}

export default App;
```

Explanation

- If `isLoggedIn` is `true`, **Welcome Back!** is displayed.
- If `isLoggedIn` is `false`, the message is not displayed.

Important: Be careful when using numbers with `&&`.

```
{count && <p>Items available</p>}
```

If `count` is `0`, React can display the number `0`.

Safer version:

```
{count > 0 && <p>Items available</p>}
```

---

7. Conditional Rendering Using Logical OR (`||`)

The `||` operator can provide a fallback value when the value on its left is falsy.

Example

```
function App() {
    const username = "";

    return (
        <div>
            <h1>{username || "Guest User"}</h1>
        </div>
    );
}

export default App;
```

Output:

```
Guest User
```

Explanation

- If `username` is a non-empty string, it is displayed.
- If `username` is an empty string, **Guest User** is displayed.

Note: `||` is commonly used for fallback values. For choosing between two JSX elements, the ternary operator is usually clearer.

Part 2: Handling More Complex Conditions

8. Conditional Rendering Using Variables

We can store JSX in a variable and render that variable.

Example

```
function App() {
    const isAdmin = true;

    let message;

    if (isAdmin) {
        message = <h1>Welcome Admin</h1>;
    } else {
        message = <h1>Welcome User</h1>;
    }

    return <div>{message}</div>;
}

export default App;
```

Explanation

- The condition decides which JSX is stored in `message`.
- The `return` statement displays the stored JSX.

This approach is useful when the rendering logic becomes longer.

---

9. Conditional Rendering Using Multiple Conditions

We can check multiple conditions using `if-else if`.

Example: Calculate a Grade

```
function App() {
    const marks = 82;
    let grade;

    if (marks >= 90) {
        grade = "A+";
    } else if (marks >= 75) {
        grade = "A";
    } else if (marks >= 60) {
        grade = "B";
    } else if (marks >= 33) {
        grade = "C";
    } else {
        grade = "Fail";
```

```
    }

    return <h1>Your Grade: {grade}</h1>;
}

export default App;
```

Output:

```
Your Grade: A
```

Explanation

React displays the grade calculated by the JavaScript conditions.

Important: Write conditions in the correct order. Check higher marks before lower marks.

---

10. Conditional Rendering Using `switch`

A `switch` statement is useful when one variable can have several possible values.

Example

```
function App() {
    const role = "teacher";
    let message;

    switch (role) {
        case "admin":
            message = "Admin Dashboard";
            break;

        case "teacher":
            message = "Teacher Dashboard";
            break;

        case "student":
            message = "Student Dashboard";
            break;

        default:
            message = "Unknown Role";
    }

    return <h1>{message}</h1>;
}
```

```
export default App;
```

Explanation

- If `role` is `"admin"`, **Admin Dashboard** is displayed.
- If `role` is `"teacher"`, **Teacher Dashboard** is displayed.
- If `role` is `"student"`, **Student Dashboard** is displayed.
- If none matches, the default message is displayed.

Part 3: Conditional Rendering with Lists and State

11. Conditional Rendering with `map()`

We can combine conditional logic with `map()` to display elements from an array.

Example: Display Student Results

```
function App() {
    const students = [
        { id: 1, name: "Amit", passed: true },
        { id: 2, name: "Riya", passed: false },
        { id: 3, name: "Jay", passed: true }
    ];

    return (
        <div>
            <h1>Student Results</h1>

            {students.map((student) => (
                <p key={student.id}>
                    {student.name} - {student.passed ? "Pass" : "Fail"}
                </p>
            ))}
        </div>
    );
}

export default App;
```

Explanation

- `map()` processes the `students` array.
- The ternary operator checks each student's result.
- `key` uniquely identifies each rendered element.

---

12. Conditional Rendering vs. `useState`

Conditional rendering decides **what should be displayed**.

`useState` allows a component to store changing data and update the UI when that data changes.

Example A: Without `useState`

```
function App() {
    const isAdmin = true;

    return (
        <div>
            {isAdmin ? <h1>Admin</h1> : <h1>User</h1>}
        </div>
    );
}

export default App;
```

The displayed result is determined by the value of `isAdmin`. Since it is set to `true`, **Admin** is displayed.

Example B: With `useState`

```
import { useState } from "react";

function App() {
    const [isAdmin, setIsAdmin] = useState(true);

    return (
        <div>
            <button onClick={() => setIsAdmin(!isAdmin)}>
                Change Role
            </button>

            {isAdmin ? <h1>Admin</h1> : <h1>User</h1>}
        </div>
    );
}

export default App;
```

Explanation

- `useState(true)` sets the initial value of `isAdmin` to `true`.
- Clicking the button changes `isAdmin` to its opposite value.
- React re-renders the component and displays the appropriate heading.

Remember: Conditional rendering does not always require `useState`. Use state when the condition needs to change during interaction.

---

13. Important Rules to Remember

1. Use `if-else` when the logic contains multiple steps.
2. Use the ternary operator when choosing between two outputs.
3. Use `&&` when displaying something only if a condition is true.
4. Use `||` for fallback values.
5. Use variables when JSX logic becomes lengthy.
6. Use `switch` when one value has multiple possible cases.
7. Use `map()` when rendering lists of elements.
8. Use a unique, stable `key` when rendering repeated elements.
9. Use `useState` when changing data should update the UI.
10. Check conditions in the correct order.

---

14. Quick Revision

Method	Purpose
`if-else`	Executes different code depending on a condition
Ternary (`? :`)	Chooses between two expressions
Logical AND (`&&`)	Displays an element when a condition is true
Logical OR (`	
Variables	Store JSX selected by conditional logic
`switch`	Selects an output from multiple possible cases
`map()`	Creates JSX elements from array items
`useState`	Stores changing state and triggers UI updates when state changes

---

Conclusion

Conditional rendering is an important React concept that allows us to display different UI elements based on conditions.

It is commonly used in:

- Login systems
- Dashboards
- Product cards

- Student results
- Notifications
- Loading screens

Practise each method separately before combining multiple conditions in a single component.

Practice Tasks

Level 1: Basic Conditions

1. Create a component that displays **"Pass"** if marks are at least 33; otherwise, display **"Fail"**.
2. Use a ternary operator to display **"Adult"** if age is 18 or above; otherwise, display **"Minor"**.
3. Use `&&` to display **"You have new notifications!"** only when `hasNotifications` is true.
4. Use `||` to display **"Guest User"** when the username is empty.

Level 2: Multiple Conditions

5. Create a grade calculator using `if-else if`.
6. Use `switch` to display a dashboard message for `"admin"`, `"teacher"`, and `"student"`.
7. Store conditional JSX in a variable and display it.

Level 3: React Lists and State

8. Create a student list that displays `"Pass"` or `"Fail"` for each student using `map()` and the ternary operator.
9. Create a button that toggles between **"Login"** and **"Logout"** using `useState`.
10. Create a product list that displays **"In Stock"** or **"Out of Stock"** based on each product's availability.

Challenge Task

Build a simple dashboard with:

- A button to toggle between Admin and User.
- A different heading for each role.
- A list of students with Pass/Fail results.
- A message that appears only when notifications are available.